Level 2

Hackathon – Fundamentals of Web Development

Student Details

Student Name	Sharuk basha s
Register Number	20925u23039
Department	Bsc ism
ID	astvu20919421
College	Shanmuga industries arts and science college Tiruvannamalai
Date of Submission	24/10/2026

1. Problem Statement

- Managing college events manually can be difficult and time-consuming. Students may face problems finding event details, registering for events, and checking available seats. Manual registration can also lead to duplicate entries and exceeding the event capacity.

- The College Event Management System is developed using HTML, CSS, and JavaScript to display event details, search and filter events, register students, prevent duplicate registrations, and manage cancellations. It uses Local Storage to save event and registration details in the browser. The system also displays registration counts and prevents new registrations when an event reaches its maximum capacity.

- This system helps students access event information easily and makes college event registration simpler and more organized.

2. Objectives

1. To display college events with titles, dates, venues, descriptions, and categories.

2. To search and filter events by name, category, and date.

3. To provide detailed information about each event.

4. To allow students to register for events with proper form validation.

5. To prevent duplicate registrations for the same event.

6. To display registered events and allow cancellations where permitted.

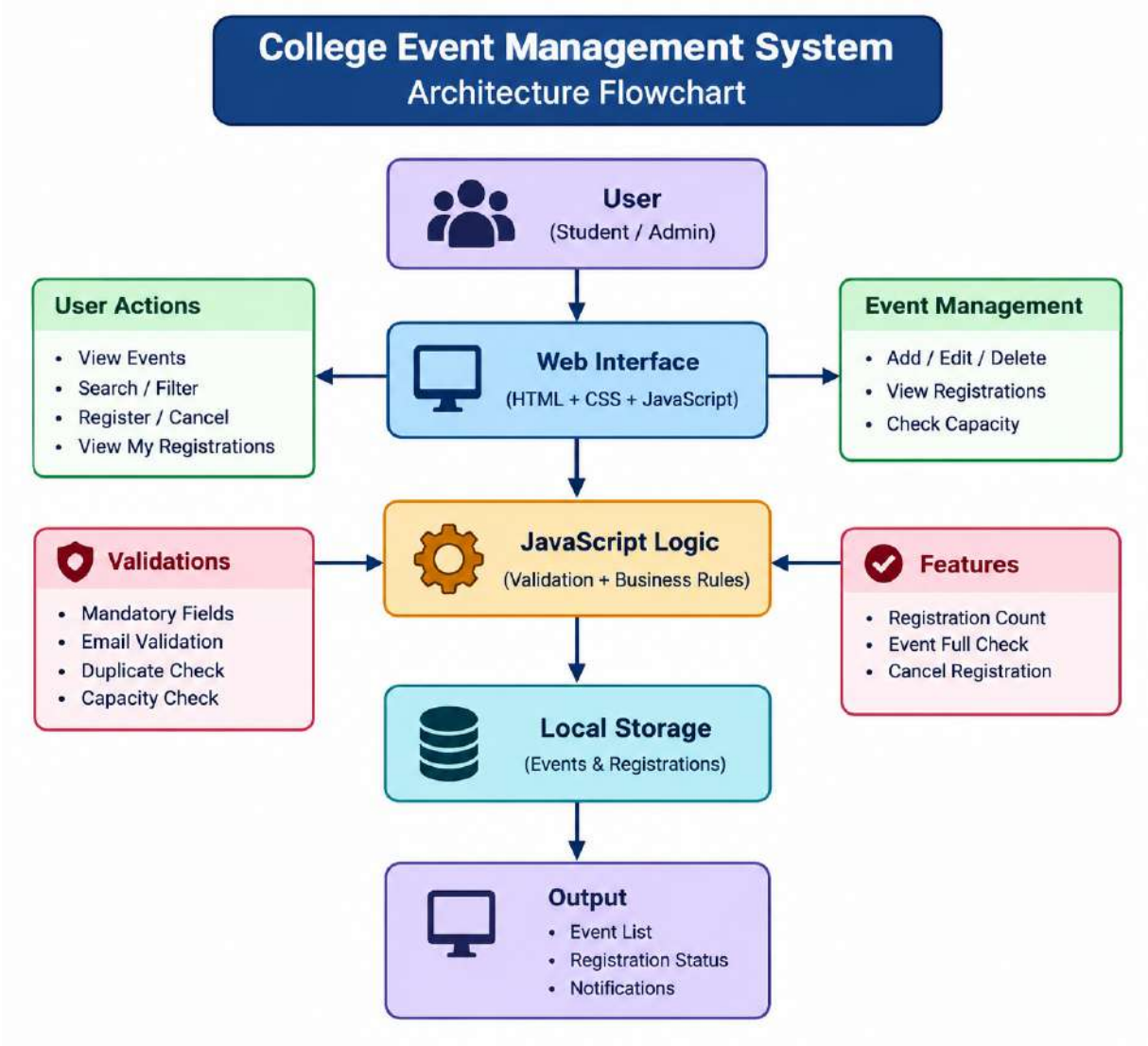
7. To store event and registration details using Local Storage.

8. To display registration counts for individual events.

9. To validate event capacity and prevent registrations when an event is full.

10. To make college event management simple, efficient, and user-friendly.

3. Architecture



Flowchart Explanation – College Event Management System

The flowchart explains the working process of the College Event Management System.

1. User: Students and admins use the system.

2. Web Interface: HTML, CSS, and JavaScript display the website and its functions.

3. JavaScript Logic: Validates user details, checks duplicate registrations, and verifies event capacity.

4. Local Storage: Stores event details and student registration information in the

browser.

5. Output: Displays event details, registration status, available seats, and notifications.

Thus, the system makes college event registration simple, organized, and user-friendly.

4. Practical Implementation

1. Event Display: Display event titles, dates, venues, descriptions, and categories dynamically.

2. Search and Filter: Search events by name and filter them by category and date.

3. Student Registration: Register students using a form with mandatory field and email validation.

4. Registration Management: Prevent duplicate registrations and allow students to cancel registrations where permitted.

5. Capacity Checking: Display registration counts and prevent registration when an event is full.

6. Data Storage: Use Local Storage to save event and registration details in the browser.

The system is implemented using HTML for structure, CSS for design, and JavaScript for functionality.

4.1 Features Implemented

1. Dynamic Event Display: Displays event titles, dates, venues, descriptions, and categories.

2. Search and Filter: Finds events by name, category, and date.

3. Event Details: Shows complete information about each event.

4. Registration Form: Validates mandatory fields and email addresses.

5. Duplicate Prevention: Prevents students from registering for the same event more than once.

6. My Registrations: Displays events registered by each student.

7. Cancellation: Allows students to cancel registrations where permitted.

8. Local Storage: Saves event and registration details in the browser.

9. Registration Count: Displays the number of registered students for each event.

10. Capacity Validation: Prevents new registrations when an event reaches its maximum capacity.

4.2 Source Code

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>College Event Management</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <h1>🎓 College Event Hub</h1>
  <p>Search, Register and Manage Events</p>
</header>

<div class="container">
  <h2>College Events</h2>

  <input id="search" placeholder="Search event name">

  <select id="category">
    <option value="">All Categories</option>
    <option>Technical</option>
    <option>Cultural</option>
    <option>Sports</option>
    <option>Workshop</option>
  </select>

  <input type="date" id="date">

  <div id="events"></div>
```

```
  <h2>Student Registration</h2>

  <form id="regForm">
    <input id="name" placeholder="Student Name" required>
    <input id="sid" placeholder="Student ID" required>
    <input id="email" type="email" placeholder="Email" required>
    <select id="eventId" required>
      <option value="">Select Event</option>
    </select>
    <button type="submit">Register</button>
  </form>

  <p id="message"></p>

  <h2>My Registrations</h2>
  <input id="myId" placeholder="Enter Student ID">
  <button type="button" onclick="showMine()">View My Events</button>
  <div id="mine"></div>
</div>

<script src="script.js"></script>
</body>
</html>
```

CSS

```
body {
  font-family: Arial;
  background: #eef2ff;
  margin: 0;
  color: #1e293b;
}

header {
```

```
  background: linear-gradient(120deg,#312e81,#9333ea);
  color: white;
  text-align: center;
  padding: 20px;
}

.container {
  max-width: 800px;
  margin: auto;
  padding: 15px;
}

input, select, button {
  width: 100%;
  padding: 12px;
  margin: 7px 0;
  border: 1px solid #ccc;
  border-radius: 7px;
  box-sizing: border-box;
}

button {
  background: #6d28d9;
  color: white;
  border: none;
  cursor: pointer;
}

button:hover {
  background: #4c1d95;
}

h2 {
  color: #6d28d9;
}

.card {
  background: white;
```

```
  padding: 15px;
  margin: 12px 0;
  border-radius: 10px;
  border-left: 5px solid #8b5cf6;
  box-shadow: 0 3px 8px #ddd;
}

#message {
  color: green;
  font-weight: bold;
}
```

JavaScript

```
let events = [
  {
    id: 1,
    name: "Web Development",
    date: "2026-11-05",
    venue: "Computer Lab",
    category: "Technical",
    description: "Learn HTML and CSS",
    capacity: 3
  },
  {
    id: 2,
    name: "Cultural Fest",
    date: "2026-11-15",
    venue: "Auditorium",
    category: "Cultural",
    description: "Dance and Music",
    capacity: 2
  },
  {
    id: 3,
    name: "AI Workshop",
    date: "2026-11-25",
    venue: "Seminar Hall",
    category: "Workshop",
    description: "Learn about AI",
    capacity: 4
  }
];

let registrations = JSON.parse(
  localStorage.getItem("eventRegs") || "[]"
);
```

```
function save() {
  localStorage.setItem(
    "eventRegs", JSON.stringify(registrations)
  );
}

function count(id) {
  return registrations.filter(
    r => r.eventId === id
  ).length;
}

function displayEvents() {
  let search = document.getElementById("search").value.toLowerCase();
  let category = document.getElementById("category").value;
  let date = document.getElementById("date").value;

  let list = events.filter(e =>
    e.name.toLowerCase().includes(search) &&
    (!category || e.category === category) &&
    (!date || e.date === date)
  );

  document.getElementById("events").innerHTML =
    list.map(e => `
      <div class="card">
        <h3>${e.name}</h3>
        <p>📅 ${e.date}</p>
        <p>📍 ${e.venue}</p>
        <p>Category: ${e.category}</p>
        <p>${e.description}</p>
        <p>Registered: ${count(e.id)} / ${e.capacity}</p>
        <button onclick="details(${e.id})">View Details</button>
      </div>
    `).join("") || "No events found.";

  document.getElementById("eventId").innerHTML =
    '<option value="">Select Event</option>' +
    events.map(e => `
      <option value="${e.id}" ${count(e.id) >= e.capacity ? "disabled" : ""}>
        ${e.name} (${count(e.id) >= e.capacity ? "FULL" : "OPEN"})
      </option>
    `).join("");
}

function details(id) {
  let e = events.find(e => e.id === id);
  alert(
    e.name + "\nDate: " + e.date +
    "\nVenue: " + e.venue +
    "\nCategory: " + e.category +
    "\nDescription: " + e.description
  );
}
```

```
document.getElementById("regForm").onsubmit = function(event) {
  event.preventDefault();

  let name = document.getElementById("name").value.trim();
  let sid = document.getElementById("sid").value.trim();
  let email = document.getElementById("email").value.trim().toLowerCase();
  let id = Number(document.getElementById("eventId").value);
  let e = events.find(e => e.id === id);
  let msg = document.getElementById("message");

  if (!name || !sid || !email || !e) {
    msg.textContent = "Fill all required fields!";
    return;
  }

  if (registrations.some(r =>
    r.eventId === id &&
    (r.sid.toLowerCase() === sid.toLowerCase() ||
     r.email === email)
  )) {
    msg.textContent = "Already registered!";
    return;
  }

  if (count(id) >= e.capacity) {
    msg.textContent = "Event is full!";
    return;
  }

  registrations.push({name, sid, email, eventId: id});
  save();
  this.reset();
  msg.textContent = "Registration successful!";
  displayEvents();
};

function showMine() {
  let sid = document.getElementById("myId").value.trim().toLowerCase();

  let list = registrations.filter(
    r => r.sid.toLowerCase() === sid && sid !== ""
  );

  document.getElementById("mine").innerHTML =
    list.map(r => {
      let e = events.find(e => e.id === r.eventId);
      return `
        <div class="card">
          <b>${r.name}</b>
          <p>${e ? e.name : "Event unavailable"}</p>
          <p>${r.email}</p>
          <button onclick="cancelReg('${r.sid}',${r.eventId})">
            Cancel Registration
          </button>
```

```
      </div>`;
    }).join("") || "No registrations found.";
}

function cancelReg(sid, eventId) {
  registrations = registrations.filter(
    r => !(r.sid === sid && r.eventId === eventId)
  );
  save();
  showMine();
  displayEvents();
}

document.getElementById("search").oninput = displayEvents;
document.getElementById("category").onchange = displayEvents;
document.getElementById("date").onchange = displayEvents;

displayEvents();
```

4.3 Project Execution

1. Create three files: "index.html", "style.css", and "script.js".

2. Write the HTML code to create the event webpage and registration form.

3. Apply CSS to improve the website design, colours, and layout.

4. Use JavaScript to display events dynamically, search and filter events, and validate student registrations.

5. Run "index.html" in a web browser to view the application.

6. Test registration, duplicate prevention, event capacity checking, and registration cancellation.

7. Verify that registration details are saved in Local Storage and remain available when the page is reopened in the same browser.

Result: The College Event Management System runs successfully as a frontend web application and helps students view events and manage their registrations.

5. Output Screenshots

STEP 1: display event dynamically with titles date venues description and categories

Total Programs
6

Active Registrations
25

Available Seats
285

🔍 Search & Filter

Search event name...

All Categories

All Status

📅 Available Programs

Web Development

Technical | OPEN

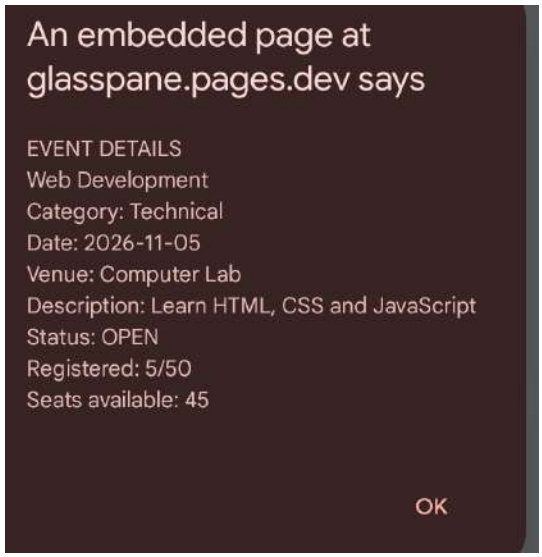
📅 2026-11-05 | 📍 Computer Lab

Learn HTML, CSS and JavaScript

Registered: 5 / 50

Available seats: 45

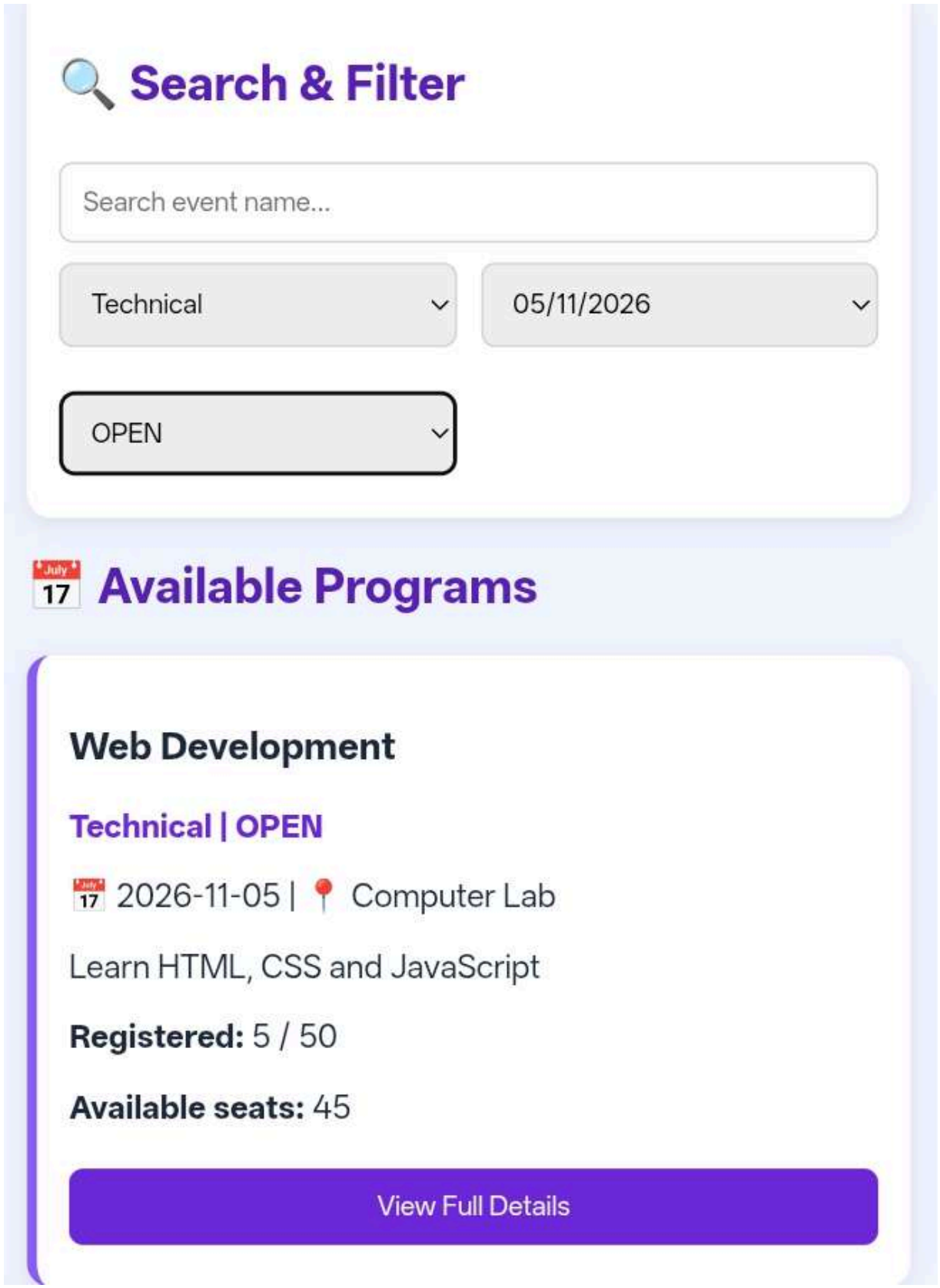
View Full Details



Explanation

The system displays college events dynamically using JavaScript. Each event contains a title, date, venue, description, and category. Instead of writing every event manually in HTML, JavaScript reads event details from an array and displays them automatically on the webpage

STEP 2: search and filter

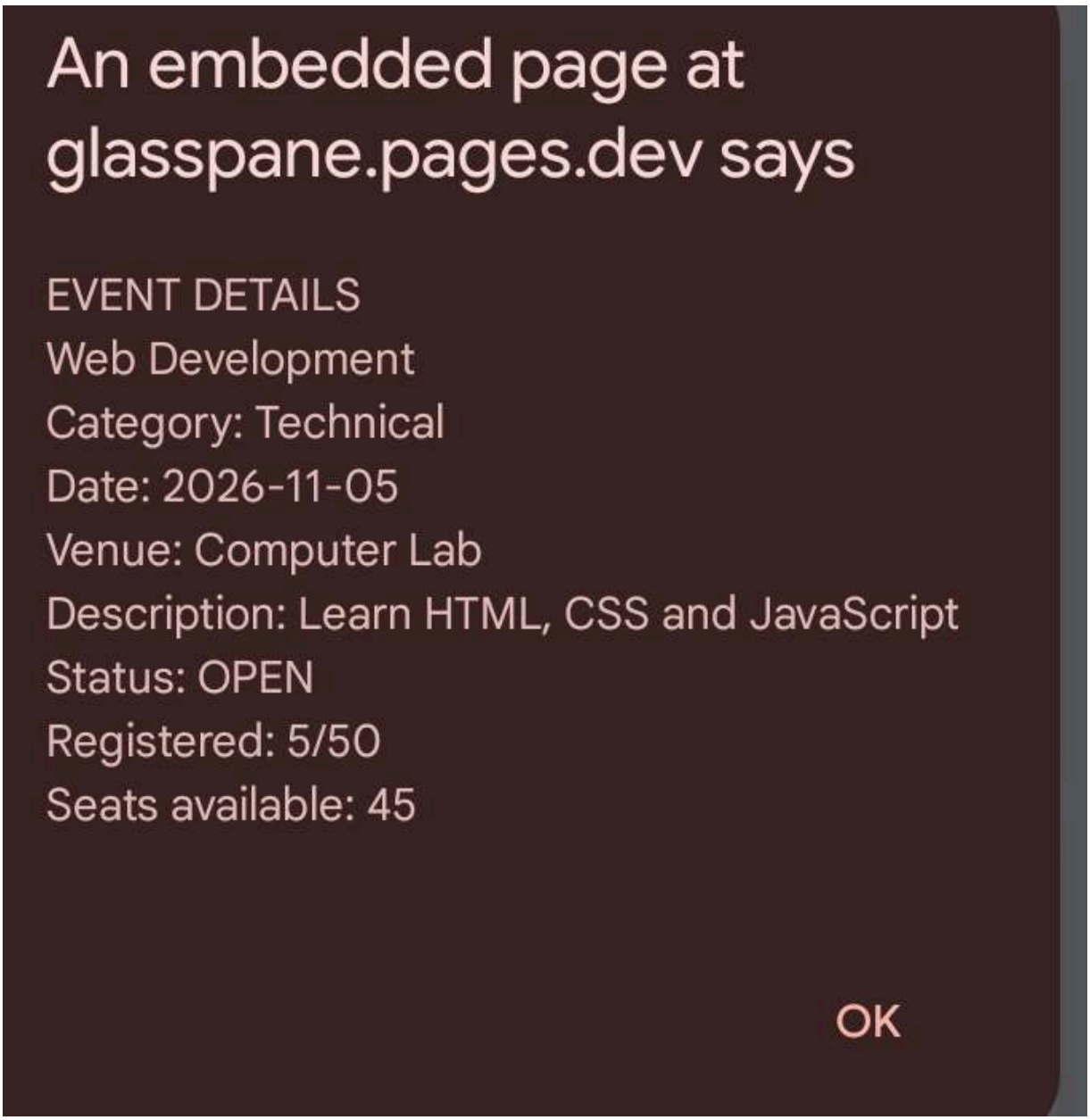


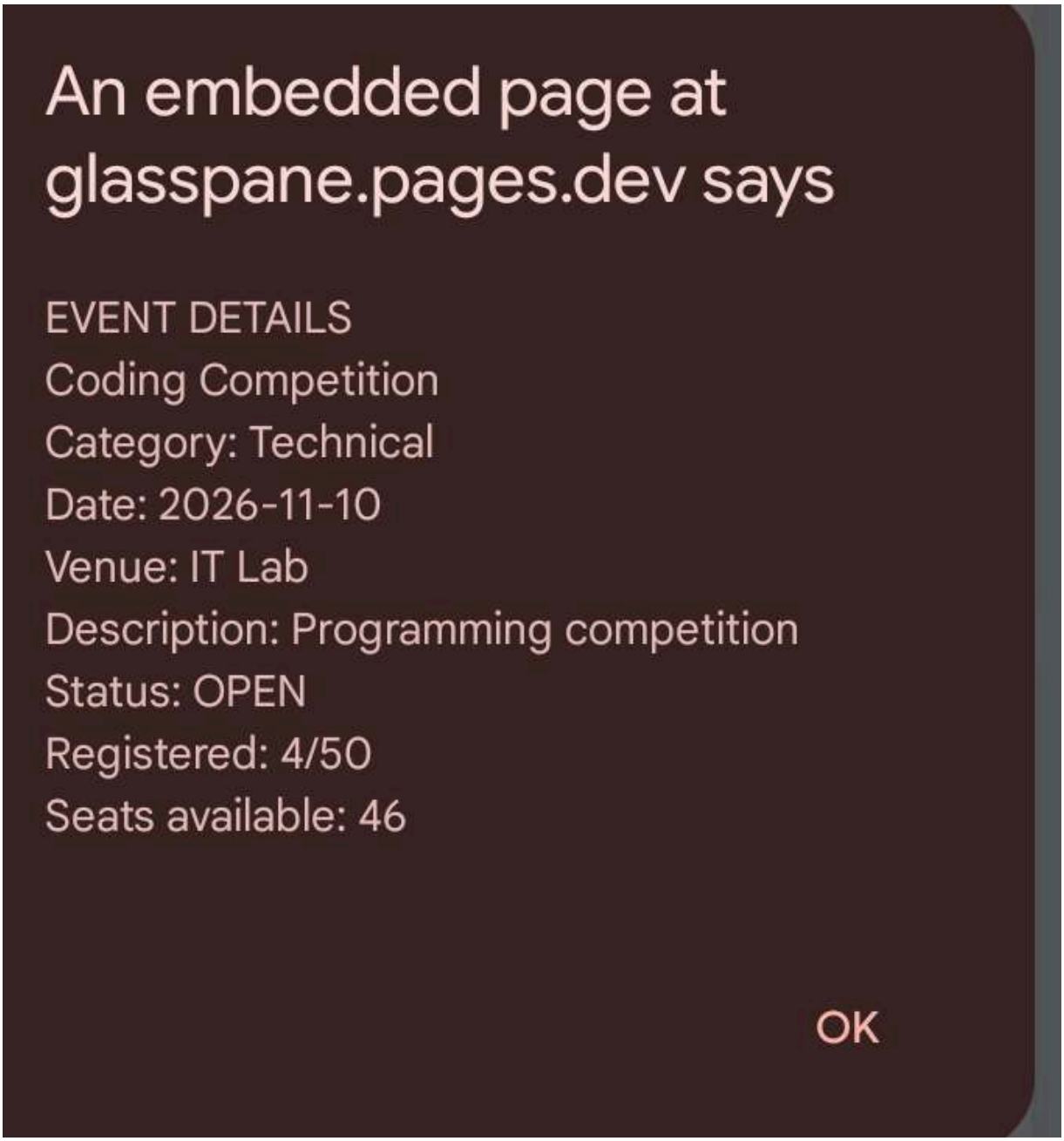
Explanation

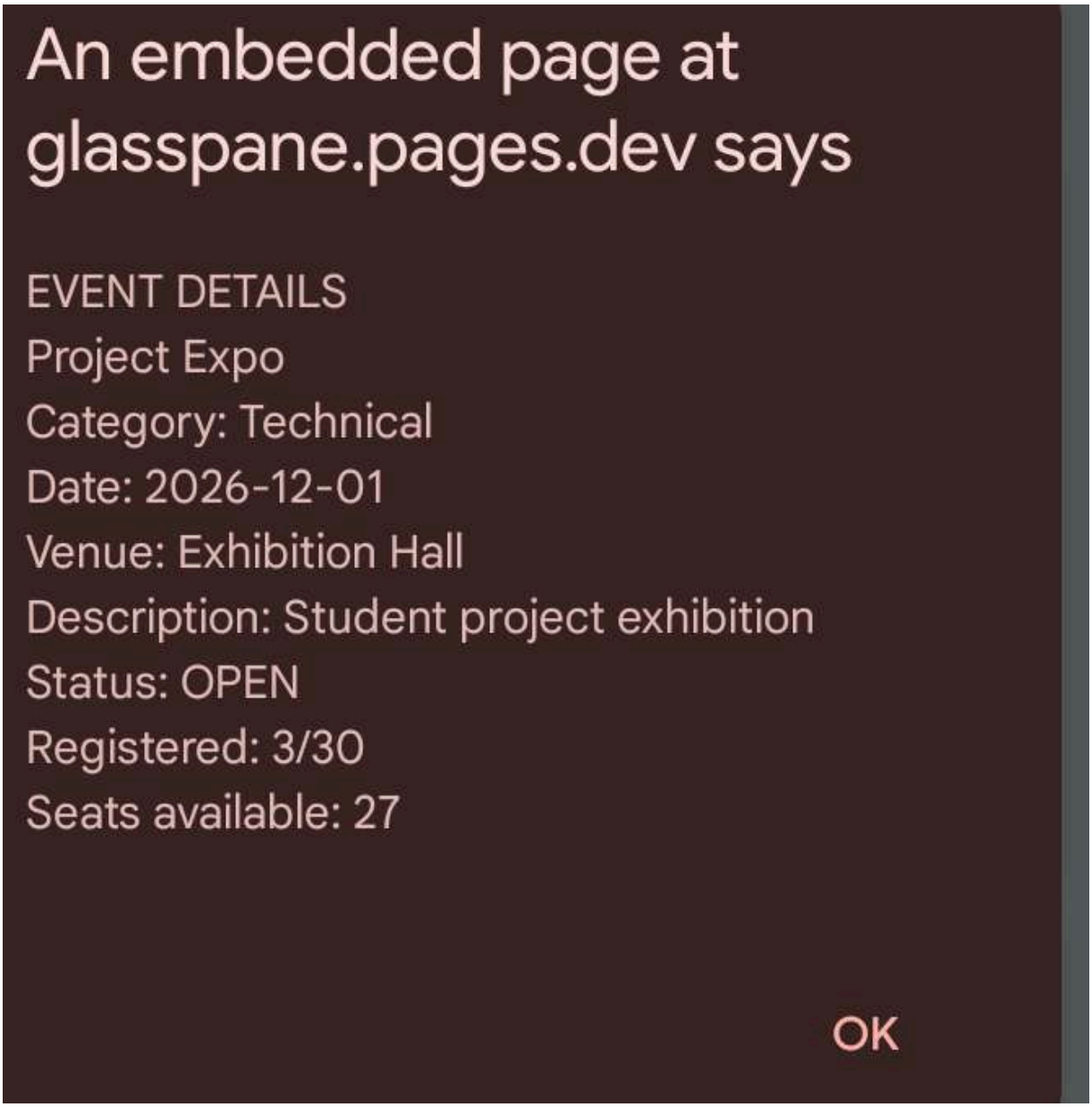
- The Search and Filter feature helps students find college events easily.

- Search by name: Finds an event using its name
- Filter by category: Displays events from a selected category, such as Technical, Cultural, or Sports.
- Filter by date: Displays events scheduled for a specific date

Step 3: display detailed information each event




An embedded page at glasspane.pages.dev says
EVENT DETAILS
Coding Competition
Category: Technical
Date: 2026-11-10
Venue: IT Lab
Description: Programming competition
Status: OPEN
Registered: 4/50
Seats available: 46
OK



Explanation

*This feature allows students to view complete information about a college event by clicking the View Details button.

The information includes:

Event title

Date

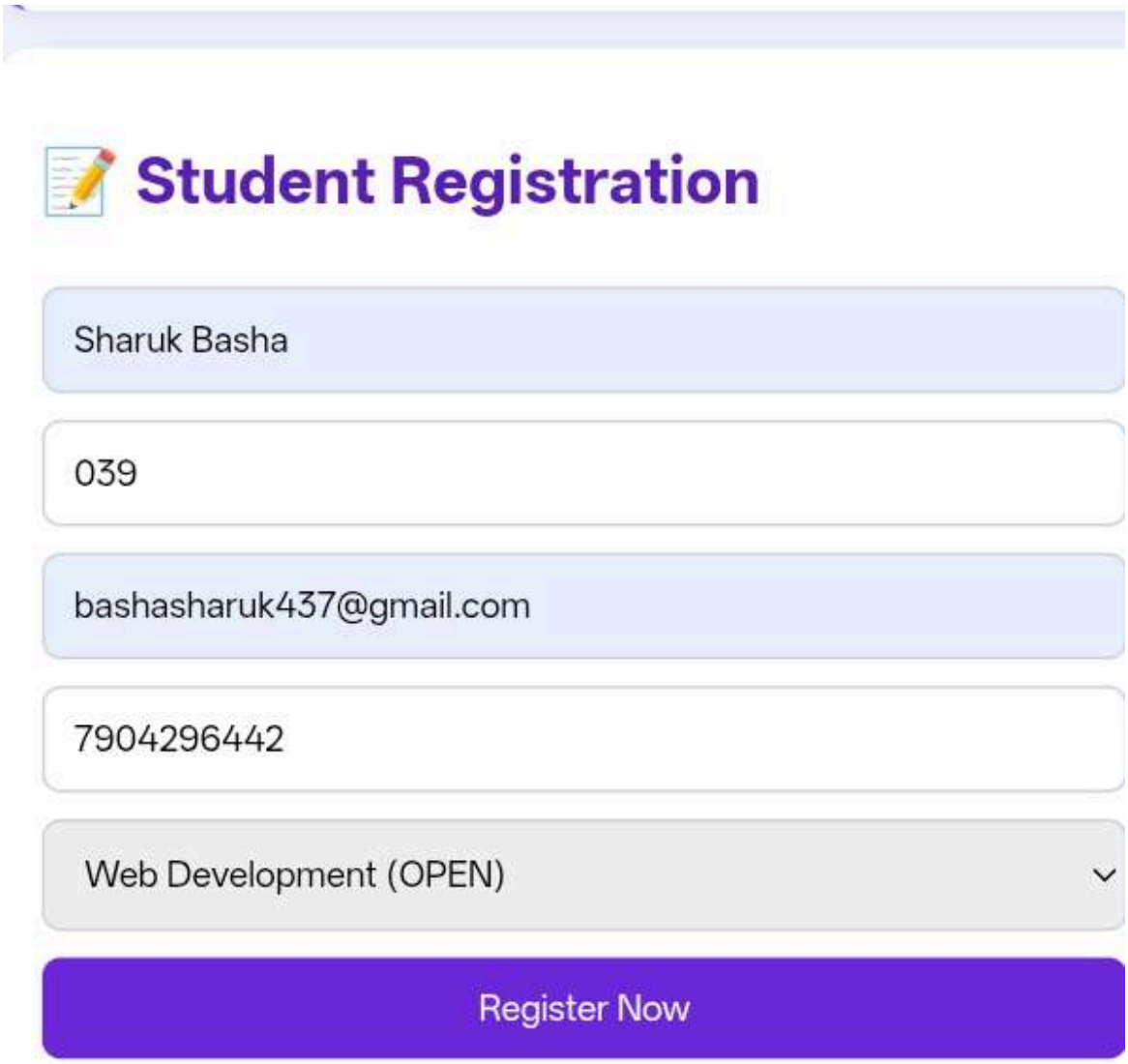
Venue

Description

Category

Registration capacity

Step 4: registration:



Explanation:

The required attribute makes important fields compulsory.

The type="email" attribute helps check whether the email address follows a valid email format.

JavaScript is used to validate the entered information. When the student clicks the Register button, the program checks the details.

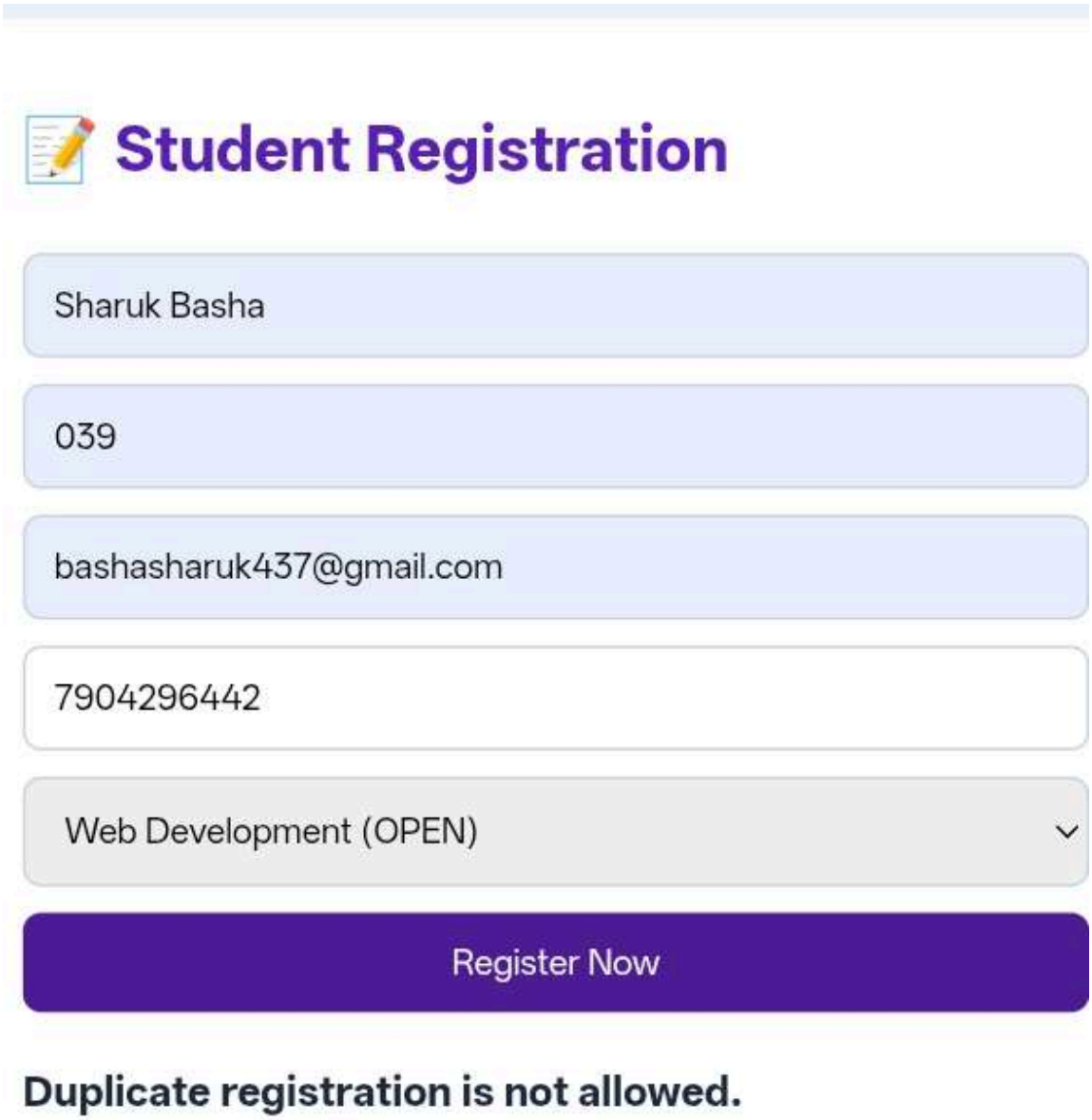
If a required field is empty, an error message is displayed. If the details pass the

checks, a registration success message appears.

The addEventListener() method detects when the form is submitted, while getElementById() helps JavaScript access the form fields. The innerText property displays the result on the webpage.

This feature helps students enter the required information correctly and makes the event registration process easier and more user-friendly.

STEP 5 :duplicate



Explanation

The Prevent Duplicate Registrations feature stops the same student from registering for the same event more than once.

For example:

Student name: Arun

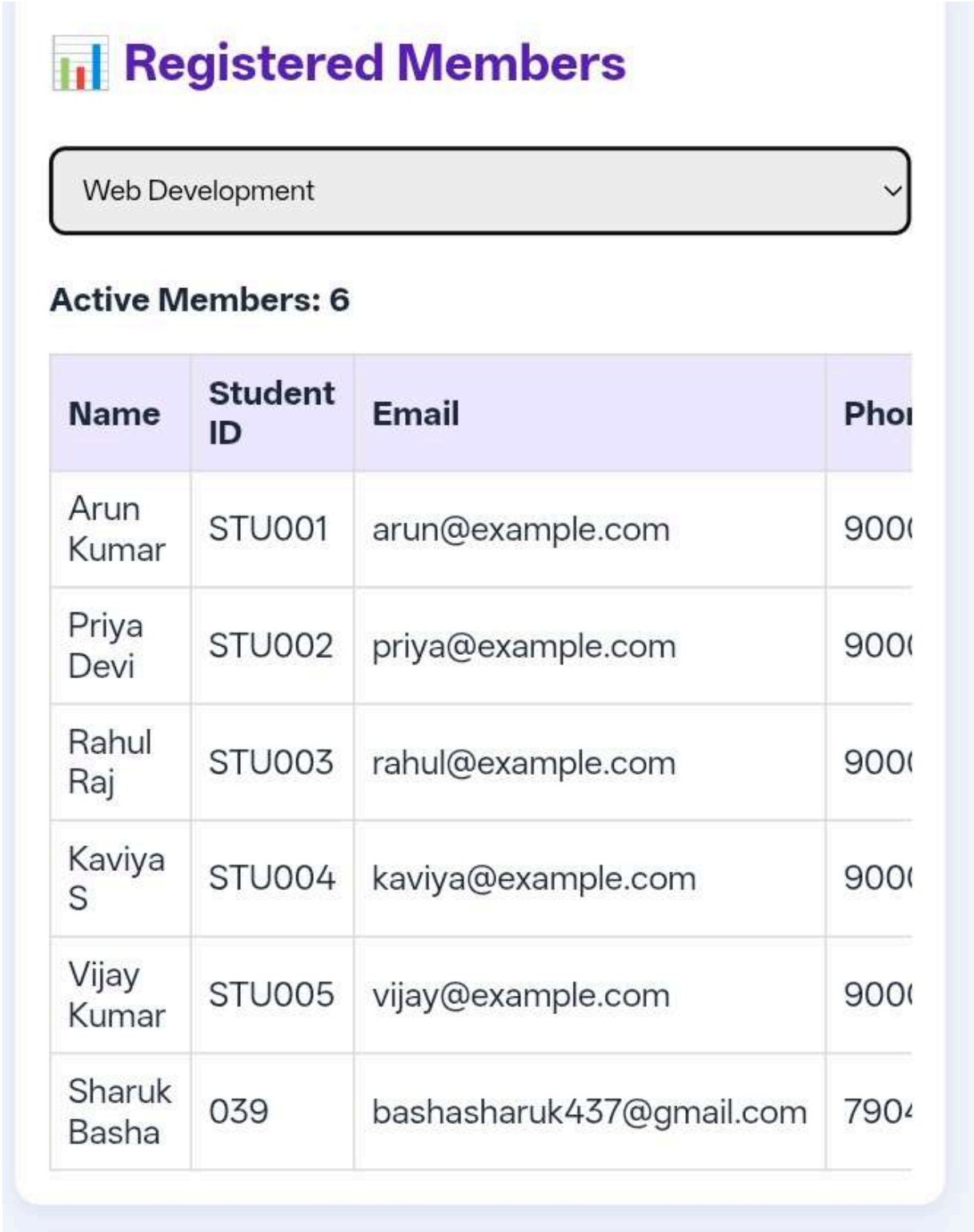
Student ID: ST101

Event: Web Development

Arun registers for the Web Development event. If Arun tries to register for the same event again using the same student ID, the system displays “You have already registered for this event!”

However, Arun can still register for a different event, such as the Cultural Fest.

Step 6: display event registration by student



Explanation

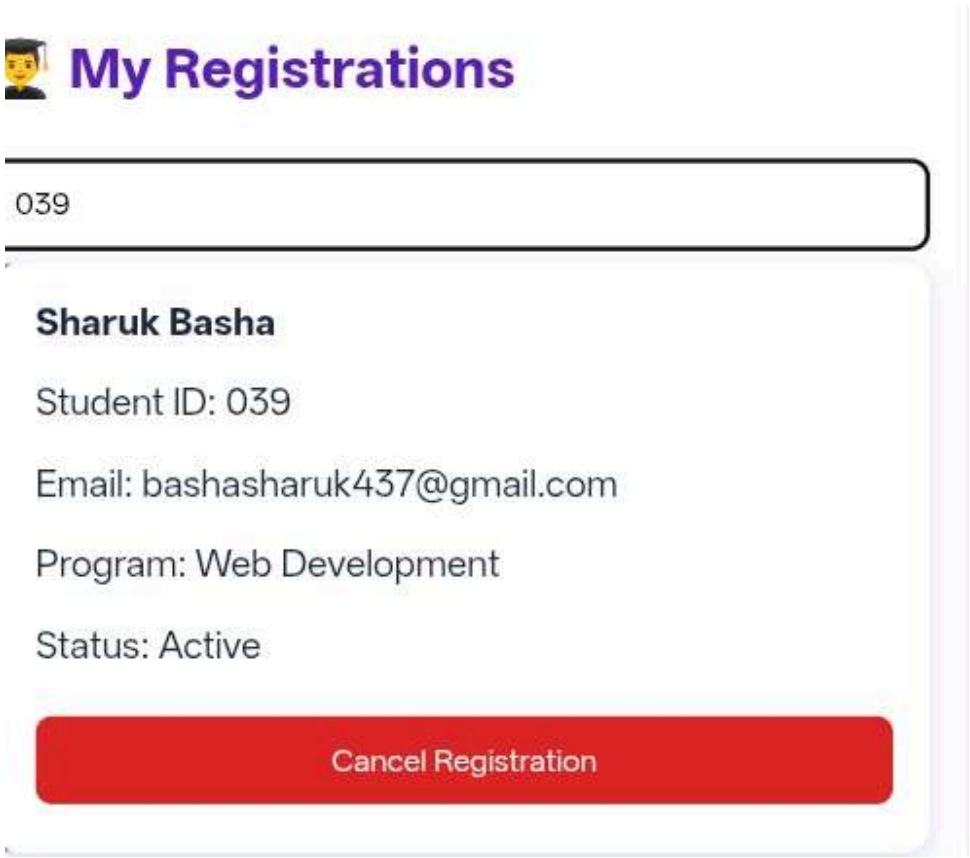
This feature allows students to see which college events they have registered for. For example, a student named Arun registers for two events:

Web Development

Cultural Fest

The system displays both events in the My Registrations section. Students can easily check their registered events without searching through the complete event list.

STEP 7: cancel



Explanation

The Cancel Event Registration feature allows students to cancel an event they have already registered for.

For example, Arun has registered for two events:

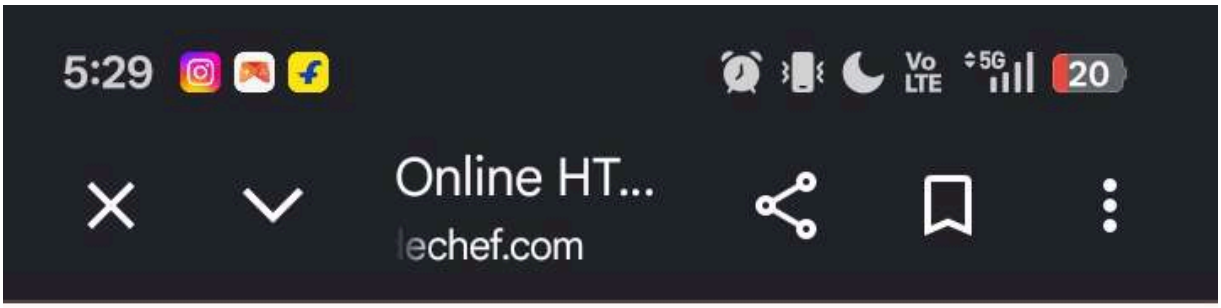
Web Development

Cultural Fest

If Arun no longer wants to attend Cultural Fest, he can click the Cancel button next to that event. The system removes the registration and updates the list.

This feature helps students manage their event registrations easily.

Step 8: local storage



Set your own goal to finish each modules

Explanation

- Local Storage is a browser feature that saves information on the device.

- Normally, when we refresh or close a webpage, information stored only in a JavaScript variable may disappear.
- Local Storage helps keep saved information even after refreshing or reopening the page, provided the browser data has not been cleared.

- For example, a student registers for a Web Development event. The system saves the registration in Local Storage. When the student reopens the webpage, the saved registration can be displayed again.

Step 9: individual events



Explanation

- This feature displays the total number of students registered for each college event.

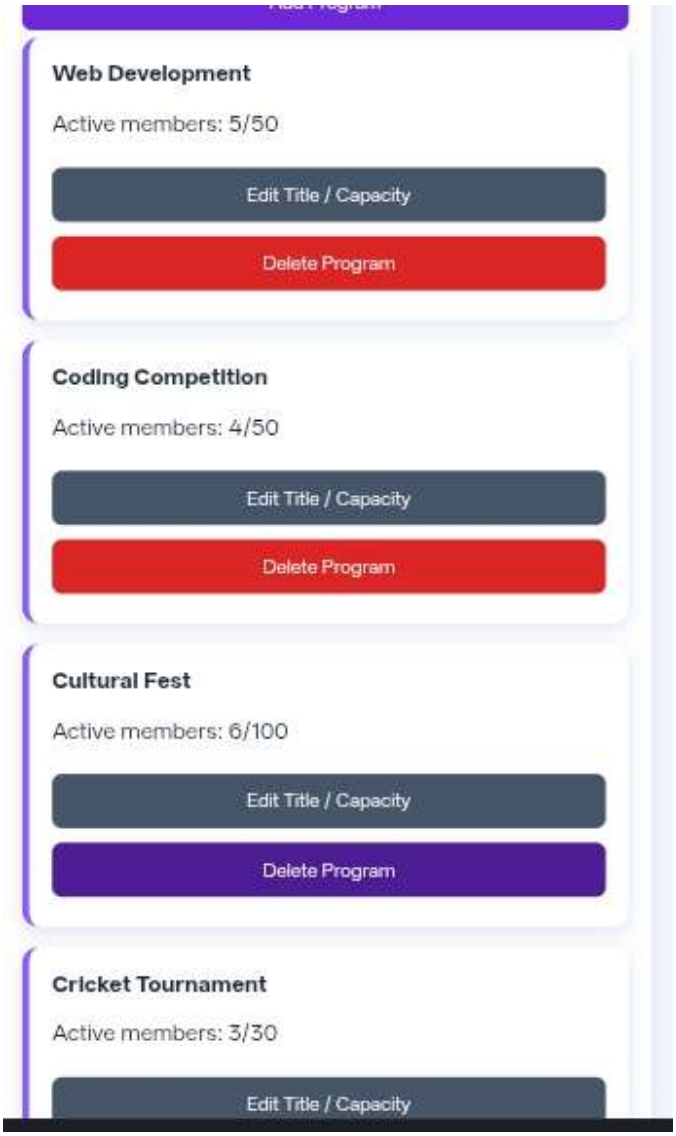
For example:
Web Development — 25 students
Cultural Fest — 40 students

AI Workshop — 15 students

The system counts the registrations and displays the total beside each event. This helps college staff understand how many students are interested in each event.

Step 10: prevent registration





Explanation

- This feature limits the number of students who can register for an event.
- For example, a Web Development Workshop has only 3 seats.
- Student 1 registers → 1/3 seats filled.
- Student 2 registers → 2/3 seats filled.
- Student 3 registers → 3/3 seats filled.
- Student 4 tries to register → Registration is rejected because the event is full.

6. Technical Explanation

- The system allows students to view their registered events and cancel registrations.
- The push() method adds new registrations, while the filter() method removes cancelled records.
- Event registration counts can be calculated from registration data, and capacity validation prevents registrations when an event reaches its maximum number of seats.
- Local Storage is used to save and retrieve registration information in the browser.
- The localStorage.setItem() method saves data, localStorage.getItem() retrieves it, JSON.stringify() converts JavaScript data into text, and JSON.parse() converts the saved text back into JavaScript data.

- Overall, the system combines HTML, CSS, and JavaScript to provide an interactive and user-friendly platform for managing college events and student registrations.
- For a production system with multiple users, a backend and database would be needed to securely manage shared registrations and event capacity.

7. Conclusion

The College Event Management System was developed using HTML, CSS, and JavaScript. It helps students view events, register, and manage their registrations easily. This project improves my web development skills and can be enhanced with login and database features in the future.

8. Future Enhancements

- QR Code Tickets: Each registered student can receive a QR-code ticket for easy event entry.
- Database Integration: A database can securely store student, event, and registration details for long-term use.
- Admin Dashboard: Administrators can add, edit, and delete events and manage student registrations.
- Mobile-Friendly Design: The website can be improved to work smoothly on mobile phones, tablets, and computers.

- Attendance Tracking: QR-code scanning can help organisers record student attendance at events.
- Event Feedback: Students can submit ratings and feedback after attending events.
- Automatic Reports: The system can generate reports showing registration counts, attendance, and popular events.
- These enhancements will make the system more secure, user-friendly, and suitable for managing large college events efficiently.